

Containerlab Python Labs — Beginner Step-by-Step Guide

This document has two sections. The first section is the full lab, from a clean Ubuntu computer to Jenkins. The second section is practice with questions and answers.

Follow the steps in order. Do not skip a failed step. Always repair the lowest layer first (Ubuntu, then Docker, then Containerlab, then ping, then BGP, then SSH, then Python, then pytest, then Jenkins).

GitHub folder used in this guide:

```
https://github.com/ajayyadav941/Network_Automation/tree/main/Containerlab_Python_Labs
```

On your computer the working directory is:

```
~/Network_Automation/Containerlab_Python_Labs
```

Older notes in the repository still say `Containerlab_Python_Labs`. That old name is wrong. Use **Containerlab_Python_Labs** everywhere.

What you will build

You will run two FRRouting (FRR) routers in Docker using Containerlab. The routers form an eBGP session, advertise loopback prefixes, and you will prove the network with ping, SSH, Netmiko, pytest, and Jenkins.

10.1.100.0/30

R1	-----	R2
FRR		FRR
AS 65001		AS 65002
Lo: 1.1.1.1/32		Lo: 2.2.2.2/32
eth1: 10.1.100.1/30		eth1: 10.1.100.2/30
Mgmt: 172.20.20.11		Mgmt: 172.20.20.12

Expected learned routes:

```
R1 learns 2.2.2.2/32 via 10.1.100.2
R2 learns 1.1.1.1/32 via 10.1.100.1
```

Words used in this lab

Word	Meaning in this lab
Ubuntu	The Linux computer (VM or laptop) where you type commands.
Docker	Runs each router as a container. The image is the blueprint. The container is the running router.
Containerlab	Reads topology/lab.clab.yml, starts the two router containers, and connects eth1 to eth1.
FRR	Free routing software. zebra handles interfaces/routes. bgpd handles BGP.
BGP	The protocol that exchanges the loopback prefixes between R1 and R2.
Management IP	172.20.20.11 and 172.20.20.12. You SSH here from Ubuntu. This is not the 10.1.100.0/30 peer link.
SSH	Remote login to a router. Netmiko uses SSH.
Netmiko	Python library that opens SSH and sends commands.
pytest	Python test runner. Six tests must pass.
JUnit XML	A results file Jenkins can display.
Jenkins	CI: clone Git, deploy the lab, run tests, publish results, destroy the lab.

What you need before you start

- Ubuntu 22.04 or 24.04 (a VM is fine). 4 GB RAM is a practical minimum. 8 GB is more comfortable.
- Internet access to install Docker, Containerlab, and Python packages, and to clone GitHub.
- A GitHub account that can read the private repository Network_Automation.
- About 20 GB free disk.

This is a classroom lab on one VM. Jenkins runs privileged with access to the host Docker socket. Do not copy that design to a production Jenkins server.

Lab password used in this repository

The Docker image is built with a password. The Python tests read the password from the `LAB_PASSWORD` environment variable, so the same value must be used when building the image and running pytest:

```
export LAB_USERNAME='root'
export LAB_PASSWORD='admin'
```

Classroom example: `admin` is used in the commands below. You may choose another password; if you do, export that same value before running Netmiko, pytest, and Jenkins.

Golden troubleshooting order

```
Host / Ubuntu
  ↓
Docker
  ↓
Containerlab
  ↓
Containers running
  ↓
Interfaces (eth1 addresses)
  ↓
Direct peer ping (10.1.100.1 <-> 10.1.100.2)
  ↓
Zebra / bgpd processes
  ↓
BGP Established (not Active, Idle, or Policy)
  ↓
BGP routes (loopbacks learned)
  ↓
Loopback ping (1.1.1.1 <-> 2.2.2.2)
  ↓
SSH to management IPs
  ↓
Netmiko
  ↓
pytest (6 passed)
  ↓
JUnit
  ↓
Jenkins
```

Part A — Ubuntu, Docker, Containerlab, and Git

Step 1. Open a terminal on Ubuntu

Log in as your normal user. Do not stay as root for the whole lab. Commands that need root use sudo.

```
whoami
hostname
pwd
```

Step 2. Update Ubuntu and install basic tools

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git curl ca-certificates python3 python3-pip python3-venv netcat-openbsd
```

Verify:

```
python3 --version
git --version
ip addr
free -h
df -h
```

You should see a Python 3 version, a Git version, at least one IP address, free memory, and free disk.

Step 3. Install Docker Engine

Use Docker’s official Ubuntu install. After install:

```
docker --version
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager
sudo docker run --rm hello-world
```

Allow your user to run Docker without sudo:

```
sudo usermod -aG docker $USER
newgrp docker
docker ps
```

Expected: docker ps prints a table and does not say “permission denied”. If it still fails, log out of Ubuntu and log in again so the docker group is applied.

Step 4. Install Containerlab

```
bash -c "$(curl -sL https://get.containerlab.dev)"
containerlab version
```

Containerlab is the tool that will start R1 and R2 from topology/lab.clab.yml.

Step 5. Clone the repository

If the repository is private, HTTPS may ask for a token. SSH is often easier after you add your public key in GitHub.

HTTPS:

```
cd ~
git clone https://github.com/ajayyadav941/Network_Automation.git
cd ~/Network_Automation/Containerlab_Python_Labs
pwd
ls
```

SSH:

```
cd ~
git clone git@github.com:ajayyadav941/Network_Automation.git
cd ~/Network_Automation/Containerlab_Python_Labs
```

You must see files such as topology/, configs/, docker/, tests/, scripts/, Jenkinsfile, requirements.txt, and Network_Testing_Containerlab.md.

```
find . -maxdepth 3 -type f | sort
```

Step 6. Understand the folder (do not skip this)

Containerlab_Python_Labs/	
├─ topology/lab.clab.yml	topology: R1, R2, link, management IPs
├─ docker/Dockerfile	FRR image with SSH
├─ configs/r1/	daemons, zebra.conf, bgpd.conf for R1
├─ configs/r2/	same files for R2
├─ lab_config.py	IPs, username, password for one Netmiko script
├─ scripts/bgp_check.py	Netmiko: R1 BGP summary
├─ scripts/Both_Router_bgp_check.py	
├─ scripts/run_tests.sh	runs pytest and writes JUnit XML
├─ tests/conftest.py	SSH fixtures for pytest
├─ tests/test_bgp.py	four BGP tests
├─ tests/test_ping.py	two loopback ping tests
├─ requirements.txt	pytest and netmiko
├─ jenkins/Dockerfile	Jenkins image for this classroom
└─ Jenkinsfile	CI pipeline

Part B — Build the FRR + SSH Docker image

The topology uses image `frr-netmiko:latest`. That image is not downloaded ready-made. You build it once from `docker/Dockerfile`. The Dockerfile starts from `frrouting/frr:latest`, installs OpenSSH, sets the root password, and starts `sshd` plus FRR.

Step 7. Set the lab password in this terminal

```
cd ~/Network_Automation/Containerlab_Python_Labs
export LAB_USERNAME='root'
export LAB_PASSWORD='admin'
```

Keep this terminal open, or export the same values again later.

Step 8. Build the image

```
docker build \
  --build-arg LAB_PASSWORD="$LAB_PASSWORD" \
  -t frr-netmiko:latest \
  -f docker/Dockerfile .
```

The last argument is a dot. It means “use this folder as the build context”. The command must be run from `Containerlab_Python_Labs`.

Verify:

```
docker images | grep frr-netmiko
```

Expected:

```
frr-netmiko    latest
```

If `LAB_PASSWORD` is empty, the build fails on purpose (the Dockerfile checks that the argument is set).

Part C — What the router configs do

You do not type these files by hand unless you are changing the lab. They are already in Git. Read them so you know what “success” looks like.

Step 9. daemons (both routers)

configs/r1/daemons and configs/r2/daemons turn on:

```
zebra=yes
bgpd=yes
staticd=yes
```

zebra owns interfaces and the routing table. bgpd speaks BGP. Other protocols stay no.

Step 10. R1 interfaces (zebra.conf)

```
hostname r1
!
interface eth1
  ip address 10.1.100.1/30
!
interface lo
  ip address 1.1.1.1/32
!
```

Step 11. R1 BGP (bgpd.conf)

```
hostname r1
!
router bgp 65001
  bgp router-id 1.1.1.1
  no bgp ebgp-requires-policy
  neighbor 10.1.100.2 remote-as 65002
!
address-family ipv4 unicast
  network 1.1.1.1/32
  neighbor 10.1.100.2 activate
exit-address-family
!
```

Step 12. R2 interfaces

```
hostname r2
!  
interface eth1  
  ip address 10.1.100.2/30  
!  
interface lo  
  ip address 2.2.2.2/32  
!
```

Step 13. R2 BGP

```
hostname r2
!  
router bgp 65002  
  bgp router-id 2.2.2.2  
  no bgp ebgp-requires-policy  
  neighbor 10.1.100.1 remote-as 65001  
  !  
  address-family ipv4 unicast  
    network 2.2.2.2/32  
    neighbor 10.1.100.1 activate  
  exit-address-family  
  !
```

no bgp ebgp-requires-policy is required in this classroom eBGP lab. Without it, FRR may show (Policy) and routes will not install.

Step 14. Topology file

topology/lab.clab.yml names the lab pytest-lab, uses management subnet 172.20.20.0/24, sets R1 to 172.20.20.11 and R2 to 172.20.20.12, bind-mounts the config files, and connects r1:eth1 to r2:eth1.

Container names after deploy:

```
clab-pytest-lab-r1  
clab-pytest-lab-r2
```

Part D — Deploy the lab

Step 15. Check that 172.20.20.0/24 is free

```
docker network ls
for n in $(docker network ls -q); do
    docker network inspect "$n" --format '{{.Name}} {{range .IPAM.Config}}{{.Subnet}}{{end}}'
done
```

If another Docker network already uses 172.20.20.0/24, you must change the management subnet and both management IPs in lab.clab.yml and in the Python files together. Freshers should keep the default if the subnet is free.

Step 16. Deploy

```
cd ~/Network_Automation/Containerlab_Python_Labs
containerlab deploy -t topology/lab.clab.yml
```

Inspect:

```
containerlab inspect -t topology/lab.clab.yml
docker ps
```

Expected management addresses: r1 172.20.20.11, r2 172.20.20.12. Expected containers: clab-pytest-lab-r1 and clab-pytest-lab-r2.

If deploy fails because the image is missing, go back to Step 8. If deploy fails because of permissions, go back to Step 3.

Part E — Manual network checks (do these before Python)

Step 17. Check eth1 addresses

```
docker exec clab-pytest-lab-r1 ip addr show eth1
docker exec clab-pytest-lab-r2 ip addr show eth1
```

Expected: R1 eth1 is 10.1.100.1/30. R2 eth1 is 10.1.100.2/30.

Step 18. Ping the peer link

```
docker exec clab-pytest-lab-r1 ping -c 4 10.1.100.2
docker exec clab-pytest-lab-r2 ping -c 4 10.1.100.1
```

Expected: 0% packet loss. Do not look at BGP until this works.

Step 19. Confirm zebra and bgpd

```
docker exec clab-pytest-lab-r1 ps aux | grep -E 'zebra|bgpd'
docker exec clab-pytest-lab-r2 ps aux | grep -E 'zebra|bgpd'
docker exec clab-pytest-lab-r1 vtysh -c "show zebra"
docker exec clab-pytest-lab-r2 vtysh -c "show zebra"
```

You may see: Can't open configuration file /etc/frr/vtysh.conf. If vtysh still prints FRR output, ignore that warning in this lab.

Step 20. BGP summary

```
docker exec clab-pytest-lab-r1 vtysh -c "show ip bgp summary"
docker exec clab-pytest-lab-r2 vtysh -c "show ip bgp summary"
```

Expected intent:

```
R1: local AS 65001, neighbor 10.1.100.2, remote AS 65002
R2: local AS 65002, neighbor 10.1.100.1, remote AS 65001
```

The neighbor must not stay in Active, Idle, or (Policy). Wait 10–20 seconds and run the command again if the session is still coming up.

Step 21. BGP routes

```
docker exec clab-pytest-lab-r1 vtysh -c "show ip route bgp"
docker exec clab-pytest-lab-r2 vtysh -c "show ip route bgp"
```

Expected:

```
R1: 2.2.2.2/32 via 10.1.100.2  
R2: 1.1.1.1/32 via 10.1.100.1
```

Step 22. Loopback ping (end-to-end)

```
docker exec clab-pytest-lab-r1 ping -c 3 2.2.2.2  
docker exec clab-pytest-lab-r2 ping -c 3 1.1.1.1
```

Expected: 0% packet loss. This is the network goal of the lab. Python only automates this proof.

Part F — SSH, Python, and Netmiko

Step 23. SSH by hand

```
ssh root@172.20.20.11
```

Password: admin (unless you changed it). First login may ask to trust the host key. Type yes.

Inside R1:

```
vtysh -c "show ip bgp summary"  
exit
```

Repeat:

```
ssh root@172.20.20.12
```

If SSH fails after you destroy and redeploy the lab, old host keys are stale:

```
ssh-keygen -R 172.20.20.11  
ssh-keygen -R 172.20.20.12
```

Port check:

```
nc -zv 172.20.20.11 22  
nc -zv 172.20.20.12 22
```

Manual SSH must work before you debug Netmiko.

Step 24. Python virtual environment

```
cd ~/Network_Automation/Containerlab_Python_Labs  
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

Verify:

```
pytest --version  
python -c "import netmiko; print(netmiko.__version__)"
```

Every new terminal must run `source .venv/bin/activate` again. The prompt usually shows `(.venv)`.

Step 25. Netmiko script for R1

```
export LAB_USERNAME='root'  
export LAB_PASSWORD='admin'  
python scripts/bgp_check.py
```

This script uses lab_config.py (host 172.20.20.11, user root, password admin). It connects with Netmiko device_type linux and runs vtysh -c "show ip bgp summary".

Expected: Connected successfully to R1, then BGP summary text.

Step 26. Netmiko script for both routers

```
python scripts/Both_Router_bgp_check.py
```

This script requires LAB_PASSWORD in the environment. Expected: BGP summary and BGP routes from R1 and R2.

Part G — pytest (six tests)

The six tests are:

1. R1 BGP neighbor is valid (AS 65001, neighbor 10.1.100.2 AS 65002, not Active/Idle/Policy)
2. R2 BGP neighbor is valid (AS 65002, neighbor 10.1.100.1 AS 65001)
3. R1 learned 2.2.2.2/32 via 10.1.100.2
4. R2 learned 1.1.1.1/32 via 10.1.100.1
5. R1 can ping 2.2.2.2 with 0% loss
6. R2 can ping 1.1.1.1 with 0% loss

tests/conftest.py opens one SSH session per router for the whole pytest session. Username defaults to root, password comes from LAB_PASSWORD, and hosts are 172.20.20.11 and 172.20.20.12.

Step 27. Run pytest

```
cd ~/Network_Automation/Containerlab_Python_Labs
source .venv/bin/activate
export LAB_USERNAME='root'
export LAB_PASSWORD='admin'
pytest -v -s tests/test_bgp.py tests/test_ping.py
```

Always name those two files. That avoids collecting leftover tests.

Expected last line: 6 passed.

Step 28. Write JUnit XML

```
mkdir -p reports
pytest -v -s tests/test_bgp.py tests/test_ping.py --junitxml=reports/results.xml
ls -lh reports/results.xml
```

Or:

```
bash scripts/run_tests.sh
```

Jenkins later reads this XML so a failed test still shows which assertion failed.

Part H — Jenkins on the same Ubuntu VM (optional CI)

Do this only after pytest already passes on the host. If pytest fails on the host, Jenkins will fail too.

Classroom design: Jenkins is a Docker container with `--privileged`, `--network host`, `--pid host`, and the host Docker socket. It copies the Git checkout to a host path, then Containerlab talks to the host Docker daemon.

Step 29. Build the Jenkins image

```
cd ~/Network_Automation/Containerlab_Python_Labs
docker build -t network-jenkins:latest -f jenkins/Dockerfile .
```

Step 30. Persistent data and a host-visible CI folder

The Jenkinsfile uses:

```
CI_DIR = '/home/jenkins/ci/network-testing'
```

If your Ubuntu username is `ajay`, create that folder. If your username is different, either create `$HOME/jenkins-ci` as well, or change `CI_DIR` in Jenkinsfile and the docker run `-v` line to your home directory. Both places must match.

```
docker volume create jenkins_home
mkdir -p $HOME/jenkins-ci
# if you are not user ajay, also:
# mkdir -p /home/$USER/jenkins-ci
# and edit Jenkinsfile CI_DIR plus the -v line below
```

Step 31. Docker group ID

```
DOCKER_GID=$(getent group docker | cut -d: -f3)
echo "$DOCKER_GID"
```

Do not guess the number. It is different on different computers.

Step 32. Start Jenkins

```
docker run -d \
  --name jenkins \
  --restart unless-stopped \
  --privileged \
  --network host \
  --pid host \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /var/run/netns:/var/run/netns \
  -v $HOME/jenkins-ci:$HOME/jenkins-ci \
  --group-add "$DOCKER_GID" \
  network-jenkins:latest
```

Do not add -p 8080:8080. Host networking already uses host port 8080.

```
docker ps
docker logs jenkins
docker exec jenkins docker ps
docker exec jenkins containerlab version
docker exec jenkins id jenkins
```

jenkins should belong to clab_admins. docker ps inside Jenkins should list containers.

Step 33. Unlock Jenkins in the browser

```
hostname -I
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Open http://YOUR_UBUNTU_IP:8080 paste the password, install suggested plugins, create an admin user. Do not put this password in Git.

Step 34. GitHub credential (private repo)

Manage Jenkins → Credentials → System → Global credentials → Add Credentials. Use a GitHub personal access token with repo read access. Do not write the token into Jenkinsfile.

Step 35. Create the Pipeline job

```
New Item → Network-Testing-Pipeline → Pipeline

Definition:      Pipeline script from SCM
SCM:             Git
Repository URL:  https://github.com/ajayyadav941/Network_Automation.git
Credentials:     the GitHub credential you added
Branch Specifier: */main
Script Path:     Containerlab_Python_Labs/Jenkinsfile
```

Script Path must be Containerlab_Python_Labs/Jenkinsfile, not the old Containerlab_Python_Labs path.

The image fr-r-netmiko:latest must already exist on the Ubuntu host (Step 8). The LAB_PASSWORD used by Jenkins credentials must match the password used when building that image.

Step 36. What the pipeline does

```
Environment Check
↓
Copy checkout to /home/jenkins/ci/network-testing
↓
Create .jenkins-venv (do not reuse your laptop .venv)
↓
Destroy any old lab
↓
Deploy topology
↓
Wait until SSH port 22 is open
↓
Wait until zebra and bgpd answer vtysh
↓
Print BGP summary and BGP routes
↓
pytest 6 tests + JUnit XML
↓
Always: publish JUnit and destroy the lab
```

A container in “running” state is not enough. SSH and FRR must be ready.

Why copy to \$HOME/jenkins-ci: Containerlab uses the host Docker daemon. Bind-mount source paths must exist on Ubuntu, not only inside Jenkins’ volume workspace.

Part I — Break BGP on purpose, then fix it

This proves pytest and Jenkins fail for a real network mistake, not because Ubuntu is broken.

Step 37. Wrong remote-AS on R2

Edit configs/r2/bgpd.conf. Change:

```
neighbor 10.1.100.1 remote-as 65001
```

to:

```
neighbor 10.1.100.1 remote-as 65100
```

Redeploy:

```
cd ~/Network_Automation/Containerlab_Python_Labs
containerlab destroy -t topology/lab.clab.yml --cleanup || true
containerlab deploy -t topology/lab.clab.yml
```

Wait for FRR, then:

```
source .venv/bin/activate
pytest -v -s tests/test_bgp.py tests/test_ping.py
```

Expected: containers up, SSH up, FRR up, BGP wrong, pytest FAIL.

Step 38. Restore

Put remote-as 65001 back. Redeploy. pytest should show 6 passed.

Part J — If something fails

Problem	What to run / what to fix
docker ps permission denied	usermod -aG docker \$USER then log out and in, or newgrp docker
containerlab deploy: image not found	Build frr-netmiko:latest from docker/Dockerfile with LAB_PASSWORD
Wrong folder after clone	cd ~/Network_Automation/Containerlab_Python_Labs
R1/R2 not running	docker ps -a; docker logs clab-pytest-lab-r1; docker logs clab-pytest-lab-r2
No eth1 address	Check binds in lab.clab.yml and configs/r1/zebra.conf
Peer ping fails	Fix Layer 3 on 10.1.100.0/30 before BGP
zebra/bgpd missing	cat /etc/frr/daemons inside the container; zebra=yes and bgpd=yes
BGP Active/Idle	Peer ping, neighbor IP, remote-AS, bgpd process
BGP (Policy)	no bgp ebgp-requires-policy on both routers
Session up, no /32 route	network statement, loopback address, address-family activate
SSH fails	nc -zv 172.20.20.11 22; ssh-keygen -R; password admin; rebuild image if password mismatch
import netmiko fails	source .venv/bin/activate; pip install -r requirements.txt
pytest 6 tests not collected	pytest -v -s tests/test_bgp.py tests/test_ping.py
Jenkins Script Path 404	Containerlab_Python_Labs/Jenkinsfile
Jenkins not in clab_admins	Rebuild jenkins/Dockerfile and recreate the container
Bind mount path missing in CI	Deploy from host-visible CI_DIR, not from jenkins_home workspace only
Git dubious ownership	sudo chown -R \$USER:\$USER ~/Network_Automation

Step 39. Fresh start for the next class

```
cd ~/Network_Automation/Containerlab_Python_Labs
containerlab destroy -t topology/lab.clab.yml --cleanup || true
docker images | grep frr-netmiko
containerlab deploy -t topology/lab.clab.yml
containerlab inspect -t topology/lab.clab.yml
```

Destroy removes running containers. It does not remove frr-netmiko:latest. You rebuild the image only when the Dockerfile or password changes.

Final checklist

- [] Docker works without sudo for your user
- [] containerlab version works
- [] frr-netmiko:latest exists (password admin)
- [] Working directory is Containerlab_Python_Labs
- [] R1 = 172.20.20.11 and R2 = 172.20.20.12
- [] 10.1.100.1 ping 10.1.100.2 succeeds
- [] zebra and bgpd are running
- [] BGP is Established (not Active, Idle, Policy)
- [] R1 has 2.2.2.2/32 via 10.1.100.2
- [] R2 has 1.1.1.1/32 via 10.1.100.1
- [] Loopback ping works both ways
- [] ssh root@172.20.20.11 works
- [] python scripts/bgp_check.py works
- [] pytest reports 6 passed
- [] reports/results.xml exists
- [] (optional) Jenkins job uses Containerlab_Python_Labs/Jenkinsfile
- [] (optional) intentional wrong AS makes pytest FAIL
- [] restored config makes pytest PASS

Practice

This section is separate from the steps. Each item has a question and then the answer.

Practice 1 — Working folder

Question: After you clone the GitHub repository, which directory must you cd into before docker build and containerlab deploy?

Answer:

```
~/Network_Automation/Containerlab_Python_Labs

Do not use the old name Containerlab_Python_Labs.
```

Practice 2 — Topology numbers

Question: Write R1 and R2 AS numbers, eth1 IPs, loopbacks, and management IPs.

Answer:

```
R1: AS 65001, eth1 10.1.100.1/30, lo 1.1.1.1/32, mgmt 172.20.20.11
R2: AS 65002, eth1 10.1.100.2/30, lo 2.2.2.2/32, mgmt 172.20.20.12
```

Practice 3 — Image versus container

Question: What is frr-netmiko:latest? What are clab-pytest-lab-r1 and clab-pytest-lab-r2? Does containerlab destroy delete the image?

Answer:

```
frr-netmiko:latest is the reusable Docker image (blueprint with FRR + SSH).
clab-pytest-lab-r1 and clab-pytest-lab-r2 are running containers.
containerlab destroy removes the containers, not the image.
```

Practice 4 — Build command

Question: Write the docker build command for the FRR SSH image.

Answer:

```
cd ~/Network_Automation/Containerlab_Python_Labs
export LAB_PASSWORD='admin'
docker build --build-arg LAB_PASSWORD="$LAB_PASSWORD" \
  -t frr-netmiko:latest -f docker/Dockerfile .
```

Practice 5 — Deploy and inspect

Question: Which two Containerlab commands start the lab and show the management IPs?

Answer:

```
containerlab deploy -t topology/lab.clab.yml
containerlab inspect -t topology/lab.clab.yml
```

Practice 6 — Order of checks

Question: A fresher sees BGP Idle. What should they prove first?

Answer:

```
Prove eth1 addresses and ping 10.1.100.1 <-> 10.1.100.2 first.
Do not debug Netmiko or Jenkins until the peer ping works.
```

Practice 7 — Expected BGP routes

Question: What route must R1 learn? What route must R2 learn?

Answer:

```
R1 learns 2.2.2.2/32 via 10.1.100.2
R2 learns 1.1.1.1/32 via 10.1.100.1
```

Practice 8 — Why (Policy) appears

Question: BGP shows (Policy). Which line is missing in bgpd.conf?

Answer:

```
no bgp ebgp-requires-policy

It must be present on both R1 and R2 in this classroom lab.
```

Practice 9 — Six pytest tests

Question: Name the six tests and the pytest command that runs only those tests.

Answer:

```
1. R1 BGP neighbor
2. R2 BGP neighbor
3. R1 learns R2 loopback
4. R2 learns R1 loopback
5. R1 pings 2.2.2.2
6. R2 pings 1.1.1.1

pytest -v -s tests/test_bgp.py tests/test_ping.py
```

Practice 10 — SSH versus management IP

Question: Which IP do you SSH to for R1? Which IP is the BGP neighbor on R1?

Answer:

```
SSH to R1: 172.20.20.11 (management)
BGP neighbor on R1: 10.1.100.2 (R2 eth1)

Do not mix these two addresses.
```

Practice 11 — Jenkins Script Path

Question: What Script Path must the Jenkins job use?

Answer:

```
Containerlab_Python_Labs/Jenkinsfile
```

Practice 12 — Why copy to \$HOME/jenkins-ci

Question: Why does Jenkins copy the project out of \$WORKSPACE before containerlab deploy?

Answer:

```
Containerlab uses the host Docker daemon.
Bind-mount files (configs/r1/...) must exist as real paths on Ubuntu.
Jenkins $WORKSPACE lives in a Docker volume, so it is the wrong host path.
The pipeline copies the checkout to a host-visible CI directory first.
```